Exercise 11: Nonlinear Programming (Lagrange + KKT + SVM)

Optimization in Machine Learning

> **Reproducibility**
>
> Produced with Python 3.10 and R 4.4.1. To reproduce, install the pinned package versions below.
>
> **Python**
>
> ```
> pip install numpy==2.0.2 matplotlib==3.10.8 cvxpy==1.7.5
> ```
>
> **R**
>
> ```
> install.packages("remotes")
> remotes::install_version("CVXR", "1.8.2")
> remotes::install_version("ggplot2", "3.5.1")
> remotes::install_version("mlr3", "1.6.0")
> ```

Problem 1: Lagrange multipliers

Solve the constrained optimization problem

$$\min_{(x,y)\in\mathbb{R}^2} \; x + 2y \quad \text{s.t.} \quad x^2 + 4y^2 = 4$$

by Lagrange multipliers.

Solution.

We know that at a minimum (x^*, y^*) the contour lines of the objective function f and the equality constraint g touch and thus it needs to hold that

$$\nabla f(x^*, y^*) = \lambda \nabla g(x^*, y^*)$$

for some Lagrange multiplier $\lambda \in \mathbb{R}$.

Hence, we get the two equalities

$$\begin{aligned} 1 &= \lambda 2x^* \\ 2 &= \lambda 8y^* \end{aligned}$$

yielding $x^* = 2y^*$.

The third equality is the equality constraint. It gives

$$\begin{aligned} & {x^*}^2 + 4{y^*}^2 = 4 \\ \Leftrightarrow \quad & 4{y^*}^2 + 4{y^*}^2 = 4 \\ \Leftrightarrow \quad & 8{y^*}^2 = 4 \\ \Leftrightarrow \quad & {y^*}^2 = \frac{1}{2}. \end{aligned}$$

Therefore, we have two candidate solutions

$$(x_1^*, y_1^*) = (\sqrt{2}, 1/\sqrt{2}) \quad \text{or} \quad (x_2^*, y_2^*) = (-\sqrt{2}, -1/\sqrt{2}).$$

Since $f(x_1^*, y_1^*) > f(x_2^*, y_2^*)$, we get the solution

$$(x^*, y^*) = (-\sqrt{2}, -1/\sqrt{2}).$$

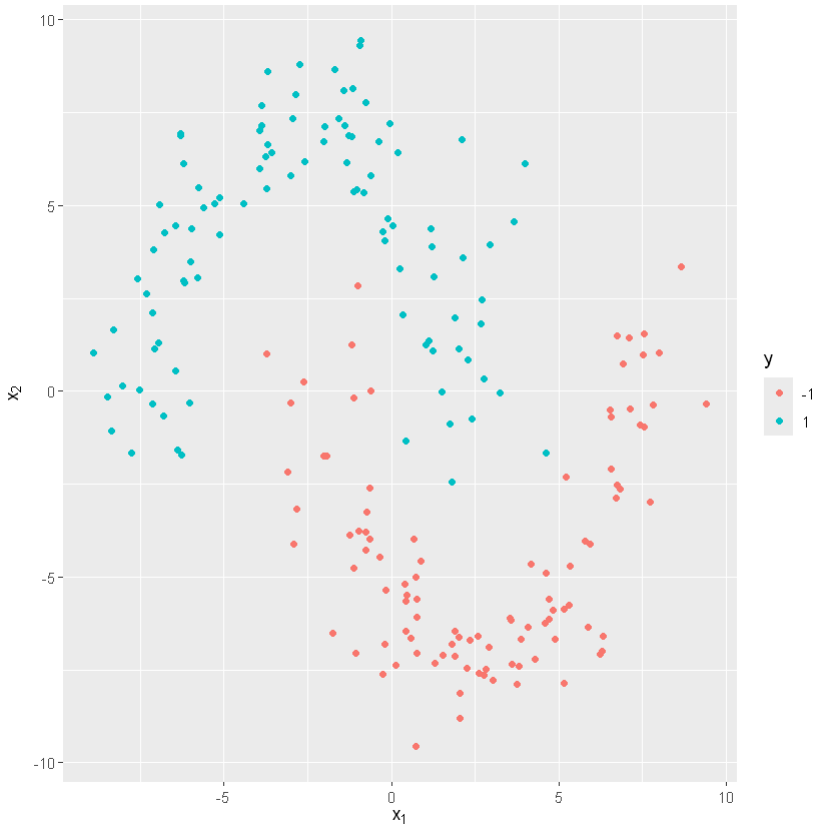
Problem 2: Nonlinear SVM

We have 200 binary observations from a *moons* distribution (two interleaved half-circles) — not linearly separable. The standard SVM trick: lift to a higher-dimensional feature space via $\phi : \mathbb{R}^d \to \mathbb{R}^l$ and learn a linear separator there. The primal soft-margin SVM in feature space is

$$\min_{\theta, \theta_0, \zeta} \ \tfrac{1}{2}\|\theta\|^2 + C \sum_{i=1}^{n} \zeta^{(i)} \quad \text{s.t.} \quad y^{(i)}\big(\langle \theta, \phi(\mathbf{x}^{(i)}) \rangle + \theta_0\big) \geq 1 - \zeta^{(i)}, \ \zeta^{(i)} \geq 0 \ \forall i.$$

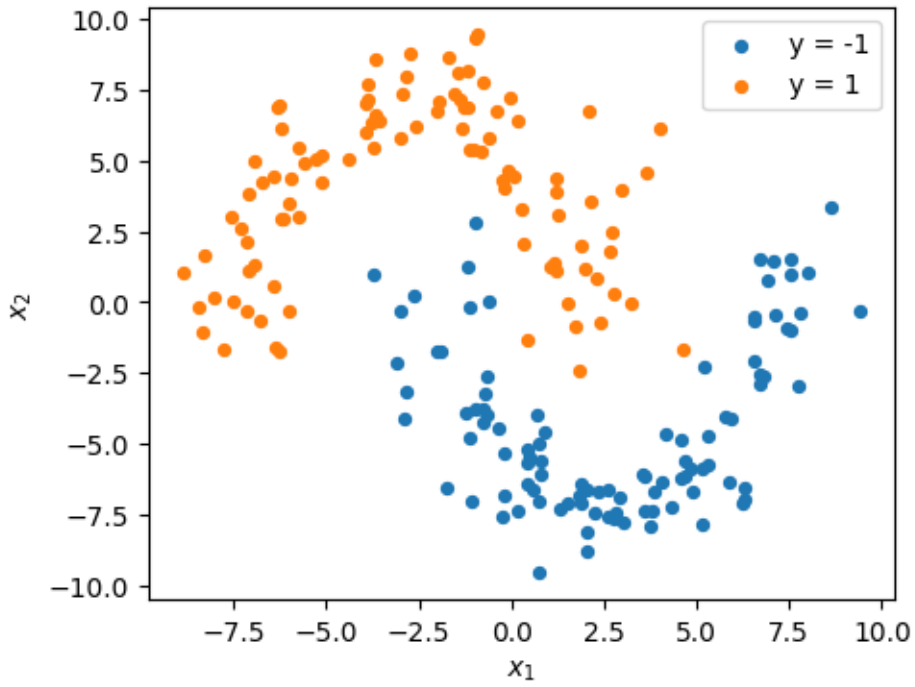
R

```
library(ggplot2)
library(CVXR)

# 200 nonlinearly separable observations from a moons distribution.
moon_data <- read.csv("data/moons.csv")
n <- nrow(moon_data)
moon_data$y_dec <- as.factor(moon_data$y)

ggplot(moon_data, aes(x = x1, y = x2, color = y_dec)) +
  geom_point(size = 1.5) +
  xlab(expression(x[1])) + ylab(expression(x[2])) +
  labs(color = expression(y))
```



Python

```
import numpy as np
import matplotlib.pyplot as plt
import cvxpy as cp

# 200 nonlinearly separable observations from a moons distribution.
data = np.genfromtxt("data/moons.csv", delimiter=",", names=True)
X = np.column_stack([data["x1"], data["x2"]])
y = data["y"].astype(float)
n = X.shape[0]

fig, ax = plt.subplots(figsize=(5, 4))
for label in (-1, 1):
    mask = y == label
    ax.scatter(X[mask, 0], X[mask, 1], label=f"y = {label}", s=18)
ax.set_xlabel(r"$x_1$"); ax.set_ylabel(r"$x_2$")
ax.legend()
plt.show()
```



(a) Lagrangian

Write down the Lagrangian of the nonlinear SVM.

Solution.

$$\mathcal{L} = 0.5\|\theta\|^2 + C\sum_{i=1}^{n}\zeta^{(i)} - \sum_{i=1}^{n}\alpha^{(i)}(\mathbf{y}^{(i)}(\phi(\mathbf{x}^{(i)})^\top\theta + \theta_0) - 1 + \zeta^{(i)}) - \sum_{i=1}^{n}\mu^{(i)}\zeta^{(i)}$$

(b) Solve the primal SVM

Solve the primal nonlinear SVM with $C = 1$ and a third-order polynomial transformation (without intercept)

$$\phi(\mathbf{x}) = (x_1, x_2, x_1^2, x_2^2, x_1x_2, x_1^2x_2, x_1x_2^2, x_1^3, x_2^3)^\top \in \mathbb{R}^9$$

using `CVXR` (R) or `cvxpy` (Python).

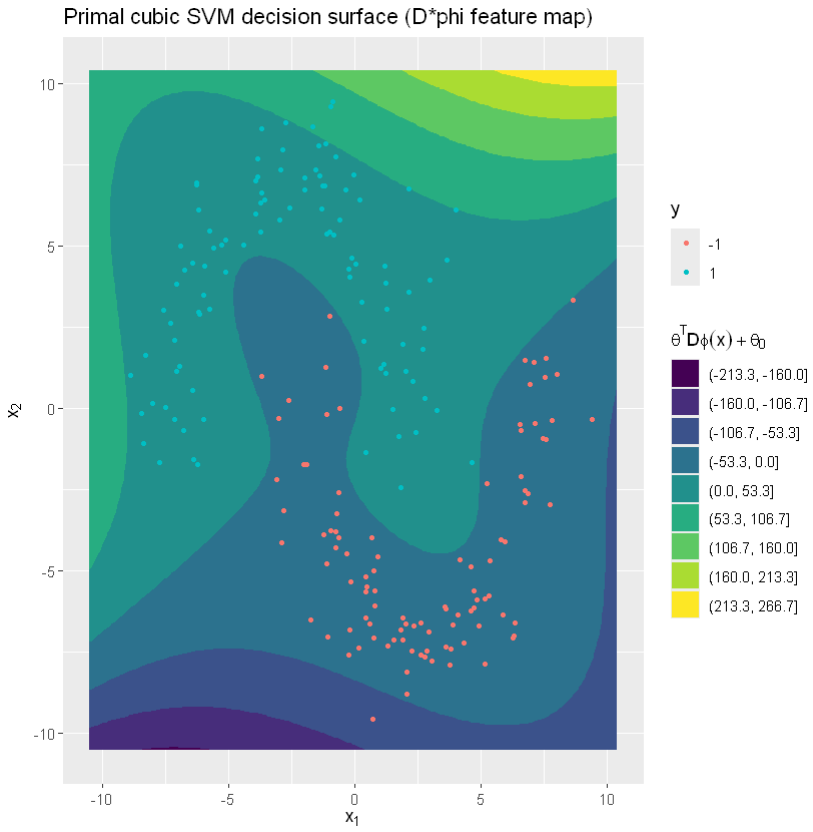
Solution.

R

```r
X <- as.matrix(moon_data[, c("x1", "x2")])

# Cubic polynomial transformation (without intercept), 9 features
cubic <- function(x) cbind(x[, 1], x[, 2],
                           x[, 1]^2, x[, 2]^2, x[, 1] * x[, 2],
                           x[, 1]^2 * x[, 2], x[, 1] * x[, 2]^2,
                           x[, 1]^3, x[, 2]^3)

# Diagonal scaling so that <D phi(x), D phi(z)> = (x^T z + 1)^3 - 1
# (matches the dual kernel in (e)).
D_diag <- c(sqrt(3), sqrt(3), sqrt(3), sqrt(3), sqrt(6), sqrt(3), sqrt(3), 1, 1)
D <- diag(D_diag)

Z   <- cubic(X)              # raw phi(X); used in (e) when recovering theta
Z_D <- Z %*% D               # D*phi(X); used in the primal constraint below
C_val <- 1.0

theta0 <- Variable()
theta  <- Variable(9)
```

```r
slack  <- Variable(n)

obj <- (1/2) * sum_squares(theta) + C_val * sum(slack)
constr <- list(
  moon_data$y * (Z_D %*% theta + theta0) >= 1 - slack,
  slack >= 0
)
prob <- Problem(Minimize(obj), constr)
solution <- solve(prob)

theta0_p <- as.numeric(solution$getValue(theta0))
theta_p  <- as.numeric(solution$getValue(theta))   # theta in D*phi basis
cat(sprintf("primal: status = %s, obj = %.4f, theta_0 = %+.4f\n",
            solution$status, solution$value, theta0_p))
cat("theta = (", paste(sprintf("%+.3f", theta_p), collapse = ", "), ")\n")

# Decision boundary on a grid spanning the data (range computed dynamically).
x_lim <- range(moon_data$x1, moon_data$x2)
pad <- 0.05 * diff(x_lim)
x_grid <- seq(x_lim[1] - pad, x_lim[2] + pad, length.out = 120)
g <- expand.grid(x1 = x_grid, x2 = x_grid)
y_g <- as.numeric(cubic(as.matrix(g)) %*% D %*% theta_p) + theta0_p
df_g <- data.frame(g, val = y_g)

ggplot() +
  geom_contour_filled(data = df_g, aes(x1, x2, z = val), bins = 10) +
  geom_point(data = moon_data, aes(x1, x2, color = y_dec), size = 1.2) +
  xlab(expression(x[1])) + ylab(expression(x[2])) +
  labs(color = expression(y),
       fill = expression(theta^T * D * phi(x) + theta[0])) +
  ggtitle("Primal cubic SVM decision surface (D*phi feature map)")
```

```
primal: status = optimal, obj = 0.4163, theta_0 = -0.1928
theta = ( +0.839, -0.295, +0.074, +0.062, +0.102, +0.013, +0.061, -0.076, +0.111 )
```

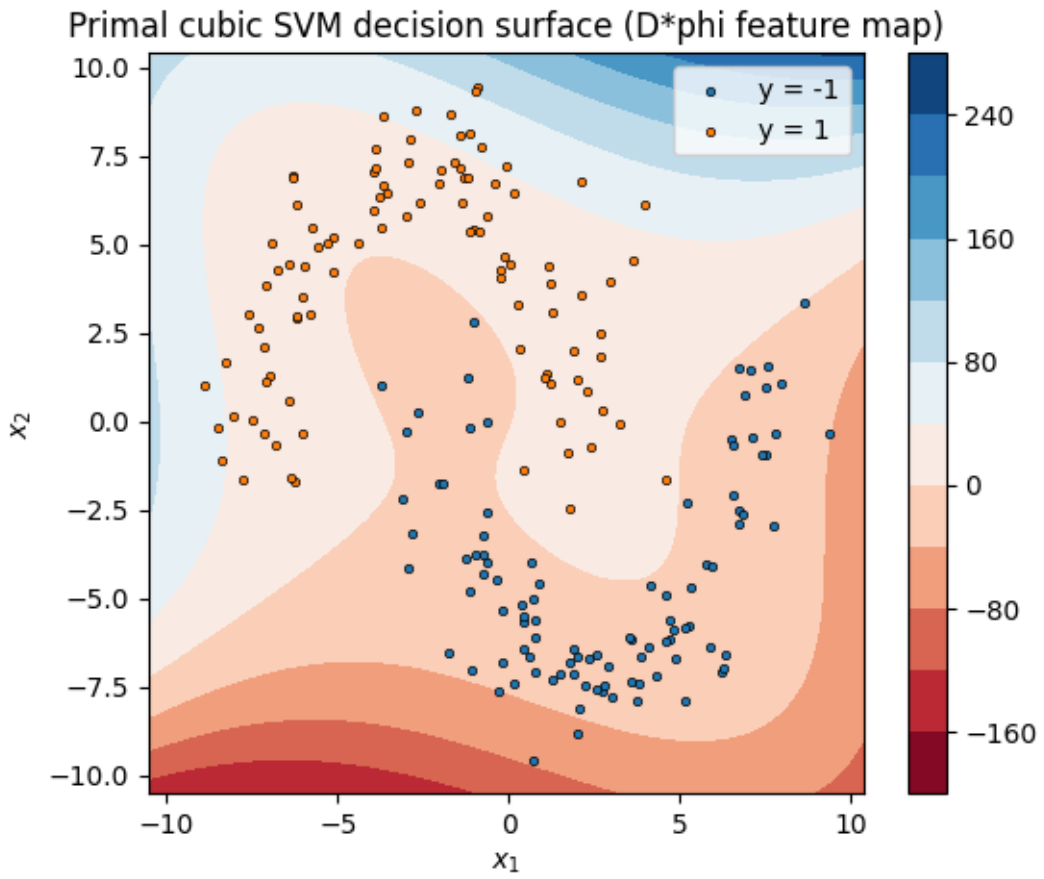


Python

```python
def cubic(X):
    """Cubic polynomial features without intercept (9 components)."""
    x1, x2 = X[:, 0], X[:, 1]
    return np.column_stack([x1, x2,
                            x1**2, x2**2, x1 * x2,
                            x1**2 * x2, x1 * x2**2,
                            x1**3, x2**3])


# Diagonal scaling so that <D phi(x), D phi(z)> = (x^T z + 1)^3 - 1
# (matches the dual kernel in (e)).
D_diag = np.array([np.sqrt(3), np.sqrt(3), np.sqrt(3), np.sqrt(6),
                   np.sqrt(3), np.sqrt(3), 1.0, 1.0])
D = np.diag(D_diag)
```

```
Z   = cubic(X)              # raw phi(X); used in (e) when recovering theta
Z_D = Z @ D                 # D*phi(X); used in the primal constraint below
C_val = 1.0

theta0 = cp.Variable()
theta  = cp.Variable(9)
slack  = cp.Variable(n, nonneg=True)

obj = cp.Minimize(0.5 * cp.sum_squares(theta) + C_val * cp.sum(slack))
constraints = [cp.multiply(y, Z_D @ theta + theta0) >= 1 - slack]
prob = cp.Problem(obj, constraints)
prob.solve()

theta0_p = float(theta0.value)
theta_p = theta.value            # theta in D*phi basis
print(f"primal: status = {prob.status}, obj = {prob.value:.4f}, "
      f"theta_0 = {theta0_p:+.4f}")
print("theta = (" + ", ".join(f"{v:+.3f}" for v in theta_p) + ")")

# Decision boundary on a grid
lo, hi = X.min(), X.max()
pad = 0.05 * (hi - lo)
g_grid = np.linspace(lo - pad, hi + pad, 120)
G1, G2 = np.meshgrid(g_grid, g_grid)
G_flat = np.column_stack([G1.ravel(), G2.ravel()])
vals = (cubic(G_flat) @ D @ theta_p + theta0_p).reshape(G1.shape)

fig, ax = plt.subplots(figsize=(6, 5))
cs = ax.contourf(G1, G2, vals, levels=10, cmap="RdBu")
for label in (-1, 1):
    mask = y == label
    ax.scatter(X[mask, 0], X[mask, 1], s=10, edgecolor="k", linewidth=0.5,
               label=f"y = {label}")
ax.set_xlabel(r"$x_1$"); ax.set_ylabel(r"$x_2$"); ax.legend()
ax.set_title("Primal cubic SVM decision surface (D*phi feature map)")
fig.colorbar(cs)
plt.show()
```

```
primal: status = optimal, obj = 0.4163, theta_0 = -0.1928
theta = (+0.839, -0.295, +0.074, +0.062, +0.102, +0.013, +0.061, -0.076, +0.111)
```



(c) KKT conditions

State the KKT conditions of the general nonlinear SVM.

Solution.

Stationarity

$$\nabla_\theta \mathcal{L} = \theta - \sum_{i=1}^{n} \alpha^{(i)} \mathbf{y}^{(i)} \phi(\mathbf{x}^{(i)}) = 0$$

$$\nabla_{\theta_0} \mathcal{L} = -\sum_{i=1}^{n} \alpha^{(i)} \mathbf{y}^{(i)} = 0$$

$$\nabla_\zeta \mathcal{L} = C \cdot \mathbf{1}_n - \alpha - \mu = 0$$

Primal feasability

$$-(\mathbf{y}^{(i)}(\phi(\mathbf{x}^{(i)})^\top\theta+\theta_0)-1+\zeta^{(i)})\leq 0$$
$$-\zeta^{(i)}\leq 0$$

Dual feasability

$$\mu\geq 0$$
$$\alpha\geq 0$$

Complementary slackness

$$-\mu^{(i)}\zeta^{(i)}=0\quad i=1,\dots,n$$
$$-\alpha^{(i)}(\mathbf{y}^{(i)}(\phi(\mathbf{x}^{(i)})^\top\theta+\theta_0)-1+\zeta^{(i)})=0\quad i=1,\dots,n$$

(d) Dual form

Derive the dual form of the nonlinear SVM. State an advantage of the dual over the primal.

Hint: use the KKT conditions to eliminate the primal variables from the Lagrangian.

Solution.

From the KKT conditions it follows that

$$\theta=\sum_{i=1}^{n}\alpha^{(i)}\mathbf{y}^{(i)}\phi(\mathbf{x}^{(i)}),$$

$$\sum_{i=1}^{n}\alpha^{(i)}\mathbf{y}^{(i)}=0,$$
$$C-\underbrace{\mu^{(i)}}_{\geq 0}=\alpha^{(i)}\Rightarrow C\geq\alpha^{(i)}\quad i=1,\dots,n.$$

Plugging these into the Lagrangian gives

$$0.5\|\sum_{i=1}^{n}\alpha^{(i)}\mathbf{y}^{(i)}\phi(\mathbf{x}^{(i)})\|^2+\sum_{i=1}^{n}\mu^{(i)}\zeta^{(i)}+\sum_{i=1}^{n}\alpha^{(i)}\zeta^{(i)}-\|\sum_{i=1}^{n}\alpha^{(i)}\mathbf{y}^{(i)}\phi(\mathbf{x}^{(i)})\|^2+\sum_{i=1}^{n}\alpha^{(i)}-\sum_{i=1}^{n}\alpha^{(i)}\zeta^{(i)})-\sum_{i=1}^{n}\mu^{(i)}\zeta^{(i)}$$

$$=-0.5\|\sum_{i=1}^{n}\alpha^{(i)}\mathbf{y}^{(i)}\phi(\mathbf{x}^{(i)})\|^2+\sum_{i=1}^{n}\alpha^{(i)}=-0.5\sum_{i=1}^{n}\sum_{j=1}^{n}\alpha^{(i)}\alpha^{(j)}\mathbf{y}^{(i)}\mathbf{y}^{(j)}\langle\phi(\mathbf{x}^{(i)}),\phi(\mathbf{x}^{(j)})\rangle+\sum_{i=1}^{n}\alpha^{(i)}$$

Hence, the dual form of the nonlinear SVM is

$$\max_{\alpha} -0.5 \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha^{(i)} \alpha^{(j)} \mathbf{y}^{(i)} \mathbf{y}^{(j)} \langle \phi(\mathbf{x}^{(i)}), \phi(\mathbf{x}^{(j)}) \rangle + \sum_{i=1}^{n} \alpha^{(i)}$$

s.t.

$$\sum_{i=1}^{n} \alpha^{(i)} \mathbf{y}^{(i)} = 0,$$
$$0 \leq \alpha \leq C.$$

Here, we can use the kernel trick to evaluate $\langle \phi(\mathbf{x}^{(i)}), \phi(\mathbf{x}^{(j)}) \rangle$ without explicitly computing the projections of each observation. (We only need to compute $\langle \mathbf{x}^{(i)}, \mathbf{x}^{(j)} \rangle$)

(e) Solve the dual SVM

Solve the dual form from (b) via `CVXR`/`cvxpy`.

Hint 1: For a polynomial transformation ϕ of order l (without intercept) there exists an invertible diagonal $\mathbf{D} \in \mathbb{R}^{l \times l}$ such that $\langle \mathbf{D}\phi(\mathbf{x}), \mathbf{D}\phi(\mathbf{z}) \rangle = (\mathbf{x}^\top \mathbf{z} + 1)^l - 1$.

Hint 2: Add $10^{-7}\mathbf{I}$ to the kernel matrix to ensure invertibility / positive-definiteness.

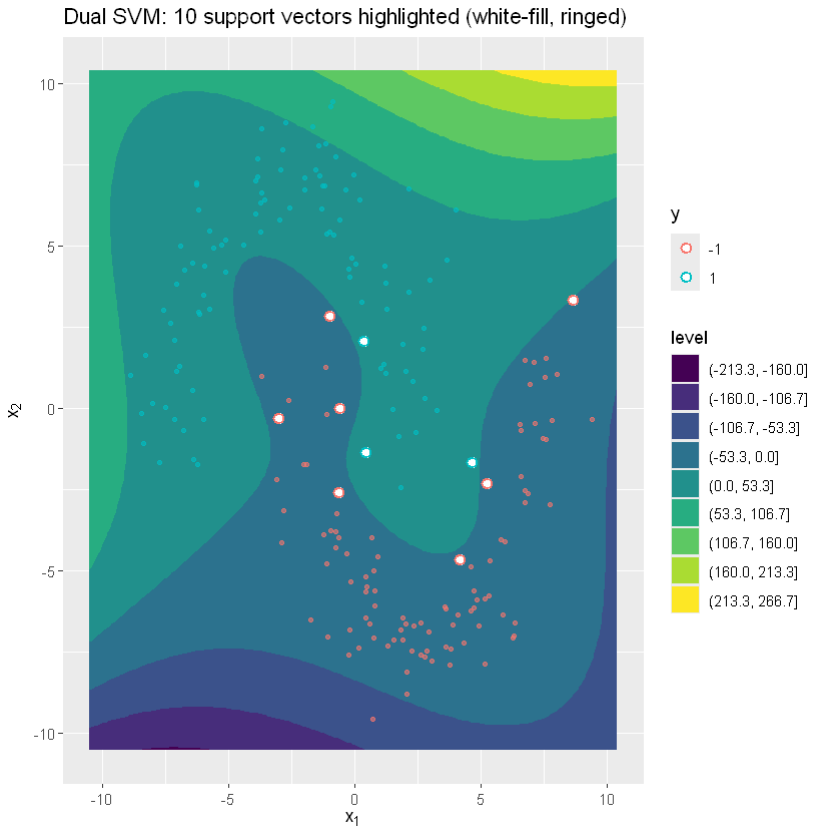
Solution.

R

```
# Kernel matrix: K = (X X^T + 1)^3 - 1.
# Hint 1: <D phi(x), D phi(z)> = (x^T z + 1)^l - 1
K <- (X %*% t(X) + 1)^3 - 1
P <- diag(moon_data$y) %*% K %*% diag(moon_data$y)
P <- P + diag(n) * 1e-7                                    # Hint 2: ensure PD

alpha <- Variable(n)
obj_d <- sum(alpha) - (1/2) * quad_form(alpha, P)
constr_d <- list(t(alpha) %*% moon_data$y == 0,
                 alpha >= 0,
                 alpha <= C_val)
prob_d <- Problem(Maximize(obj_d), constr_d)
solution_d <- solve(prob_d)
alpha_p <- as.numeric(solution_d$getValue(alpha))
```

```r
cat(sprintf("dual:   status = %s, obj = %.4f\n",
            solution_d$status, solution_d$value))

# Recover theta via the diagonal scaling D so that <D phi, D phi> = K
D <- diag(c(rep(sqrt(3), 4), sqrt(6), rep(sqrt(3), 2), 1, 1))
theta_d <- as.numeric(t(alpha_p * moon_data$y) %*% Z %*% D)

# theta_0: midpoint between max negative and min positive class scores
scores <- as.numeric(Z %*% D %*% theta_d)
theta0_d <- -0.5 * (max(scores[moon_data$y == -1]) +
                    min(scores[moon_data$y == 1]))
cat(sprintf("recovered: theta_0 = %+.4f, primal/dual obj diff = %.4e\n",
            theta0_d, solution$value - solution_d$value))

# Support vectors: alpha_i > eps
# (active margin constraint, by complementary slackness).
sv_eps <- 1e-4
sv_mask <- alpha_p > sv_eps
cat(sprintf("support vectors: %d / %d (alpha_i > %.0e)\n",
            sum(sv_mask), n, sv_eps))

y_g_d <- as.numeric(cubic(as.matrix(g)) %*% D %*% theta_d + theta0_d)
df_g_d <- data.frame(g, val = y_g_d)
moon_data$is_sv <- sv_mask

ggplot() +
  geom_contour_filled(data = df_g_d, aes(x1, x2, z = val), bins = 10) +
  geom_point(data = moon_data[!sv_mask, ], aes(x1, x2, color = y_dec),
             size = 1.0, alpha = 0.6) +
  geom_point(data = moon_data[sv_mask, ], aes(x1, x2, color = y_dec),
             size = 2.5, shape = 21, fill = "white", stroke = 1.0) +
  xlab(expression(x[1])) + ylab(expression(x[2])) +
  labs(color = expression(y)) +
  ggtitle(sprintf(paste0("Dual SVM: %d support vectors highlighted ",
                         "(white-fill, ringed)"), sum(sv_mask)))
```

```
dual:   status = optimal, obj = 0.4163
recovered: theta_0 = -0.1928, primal/dual obj diff = 7.0105e-09
support vectors: 10 / 200 (alpha_i > 1e-04)
```

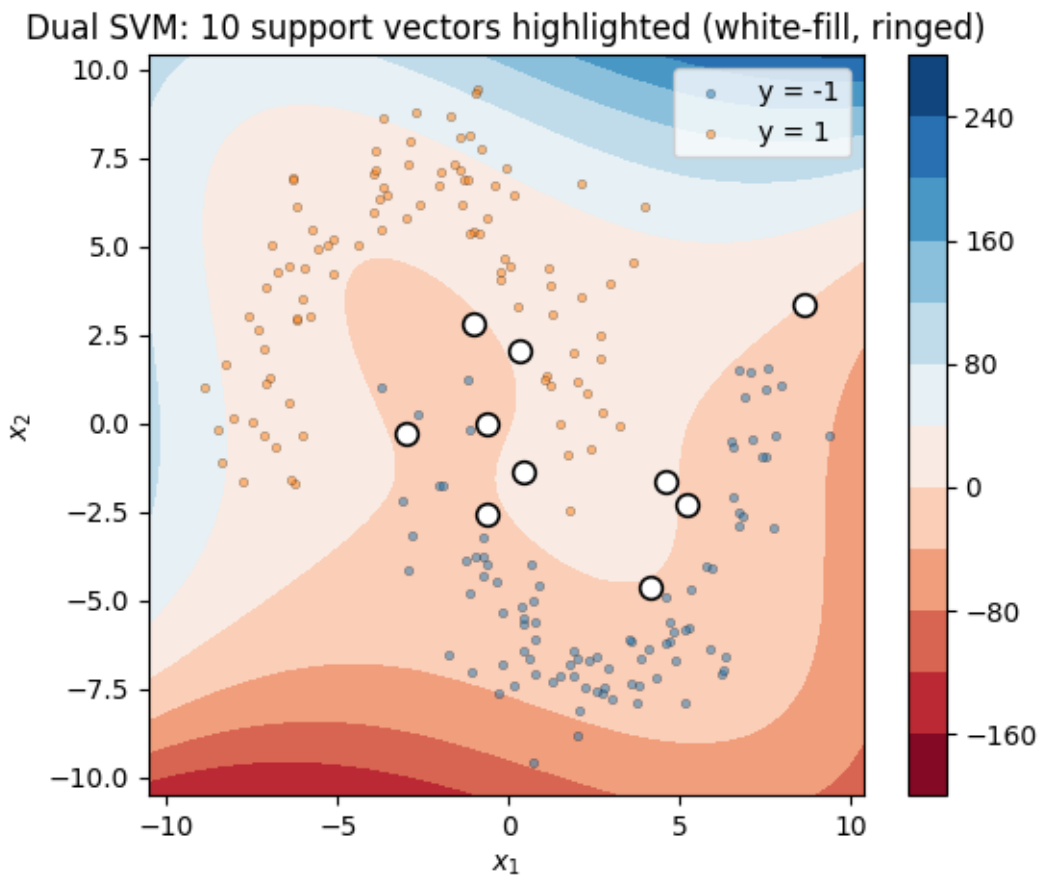


Python

```
# Kernel: K = (X X^T + 1)^3 - 1.
# Hint 1: <D phi(x), D phi(z)> = (x^T z + 1)^l - 1
K = (X @ X.T + 1) ** 3 - 1
P = np.diag(y) @ K @ np.diag(y) + 1e-7 * np.eye(n)        # Hint 2: ensure PD

alpha = cp.Variable(n, nonneg=True)
obj_d = cp.Maximize(cp.sum(alpha) - 0.5 * cp.quad_form(alpha, cp.psd_wrap(P)))
constraints_d = [alpha @ y == 0, alpha <= C_val]
prob_d = cp.Problem(obj_d, constraints_d)
prob_d.solve()

alpha_p = alpha.value
print(f"dual:   status = {prob_d.status}, obj = {prob_d.value:.4f}")

# Recover theta via the diagonal scaling D matching <D phi, D phi> = K
```

```
D = np.diag([np.sqrt(3), np.sqrt(3), np.sqrt(3), np.sqrt(3), np.sqrt(6),
             np.sqrt(3), np.sqrt(3), 1.0, 1.0])
theta_d = (alpha_p * y) @ Z @ D

scores = Z @ D @ theta_d
theta0_d = -0.5 * (scores[y == -1].max() + scores[y == 1].min())
print(f"recovered: theta_0 = {theta0_d:+.4f}, "
      f"primal/dual obj diff = {prob.value - prob_d.value:.4e}")

# Support vectors: alpha_i > eps
# (active margin constraint, by complementary slackness).
sv_eps = 1e-4
sv_mask = alpha_p > sv_eps
print(f"support vectors: {int(sv_mask.sum())} / {n} (alpha_i > {sv_eps:.0e})")

vals_d = (cubic(G_flat) @ D @ theta_d + theta0_d).reshape(G1.shape)
fig, ax = plt.subplots(figsize=(6, 5))
cs = ax.contourf(G1, G2, vals_d, levels=10, cmap="RdBu")
for label in (-1, 1):
    mask_class = y == label
    # Non-support-vector points: faded
    mask_non_sv = mask_class & ~sv_mask
    ax.scatter(X[mask_non_sv, 0], X[mask_non_sv, 1], s=10,
               edgecolor="k", linewidth=0.3, alpha=0.5, label=f"y = {label}")
    # Support vectors: ringed
    mask_sv = mask_class & sv_mask
    ax.scatter(X[mask_sv, 0], X[mask_sv, 1], s=70,
               facecolor="white", edgecolor="k", linewidth=1.2)
ax.set_xlabel(r"$x_1$"); ax.set_ylabel(r"$x_2$"); ax.legend()
ax.set_title(f"Dual SVM: {int(sv_mask.sum())} support vectors "
             f"highlighted (white-fill, ringed)")
fig.colorbar(cs)
plt.show()
```

```
dual:   status = optimal, obj = 0.4163
recovered: theta_0 = -0.1928, primal/dual obj diff = 7.0105e-09
support vectors: 10 / 200 (alpha_i > 1e-04)
```


Dual SVM: 10 support vectors highlighted (white-fill, ringed)
y = -1
y = 1
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−10.0
−10
−5
0
5
10
x_1
x_2
240
160
80
0
−80
−160